

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	3
2.1	Topics	3
3	Class Index	5
3.1	Class List	5
4	File Index	7
4.1	File List	7
5	Topic Documentation	9
5.1	namespace untangle: functions	9
5.1.1	Detailed Description	9
5.1.2	Function Documentation	9
5.1.2.1	bind() [1/2]	9
5.1.2.2	bind() [2/2]	10
6	Class Documentation	11
6.1	untangle::async::execution< actionT > Class Template Reference	11
6.1.1	Detailed Description	11
6.1.2	Member Function Documentation	12
6.1.2.1	bind_action_and_method()	12
6.1.2.2	is_running()	12
6.1.2.3	add_action()	12
6.2	untangle::async::execution_poll Class Reference	13
6.2.1	Detailed Description	13
6.2.2	Member Function Documentation	13
6.2.2.1	add()	13
6.2.2.2	is_running()	14
6.2.2.3	get()	14
7	File Documentation	15
7.1	async.hpp File Reference	15
7.1.1	Detailed Description	15
7.2	async.hpp	16
	Index	19

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type
    untangle::async::execution<std::function<void(void)>> execution;
    std::function<void(void)> action;
    // create async binding between the action and f
    execution.bind_action_and_function(action, f);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    action();
    // run the execution
    execution.run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}
```

example: one-off execution of a class method

```
#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type
    untangle::async::execution<std::function<void(int)>> execution;
    execution.bind_action_and_method(a->action, a, &A::f); // note the bound object must be a shared pointer
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution.run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}
```

continuous execution



example: continuous execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type
    untangle::async::execution<std::function<void(void)>> execution;
    std::function<void(void)> action;
    // create async binding between the action and f
    execution.bind_action_and_function(action, f);

    // run the execution
    execution.start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution.stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}
```

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [9](#)

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::execution< actionT >	
Asynchronous template execution class	11
untangle::async::execution_poll	
Execution poll class	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp	Interface to untangle::async::execution class	15
---------------------------	---	----

Chapter 5

Topic Documentation

5.1 namespace untangle: functions

Functions

- `template<typename classT, typename T, typename actionT = std::function<T>>>
actionT untangle::async::bind (const std::shared_ptr< classT > &obj, T classT::*method,
execution< actionT > &async_exec)`
Asynchronous binding to a class function member.
- `template<typename T, typename actionT = std::function<T>>>
actionT untangle::async::bind (T &Fn, execution< actionT > &async_exec)`
Asynchronous binding to a plain function.

5.1.1 Detailed Description

5.1.2 Function Documentation

5.1.2.1 `bind()` [1/2]

```
template<typename classT, typename T, typename actionT = std::function<T>>>  
actionT untangle::async::bind (  
    const std::shared_ptr< classT > & obj,  
    T classT::* method,  
    execution< actionT > & async_exec)
```

Asynchronous binding to a class function member.

It uses an [execution](#) object to execute the bound class function member on other thread. It creates an action as a binding to a class method (by `untangle::bind()`), and returns a callable that passes this action to [execution::add_action\(\)](#).

Parameters

obj	- A std::shared_ptr that wraps the bound class object.
-----	--

method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- An execution object.

Returns

- A `std::function<...>(lambda)` that adds the action to the execution object's actions list.

5.1.2.2 `bind()` [2/2]

```
template<typename T, typename actionT = std::function<T>>>
actionT untangle::async::bind (
    T & Fn,
    execution< actionT > & async_exec)
```

Asynchronous binding to a plain function.

It uses an [execution](#) object to execute the bound function on other thread. It creates an action as `std::function` that wraps the function, and returns a callable that passes this action to [execution::add_action\(\)](#).

Parameters

Fn	- A plain function.
async_exec	- An execution object.

Returns

- A `std::function(lambda)` that adds the action to the execution object's actions list.

Chapter 6

Class Documentation

6.1 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- `execution ()`
Constructs a new execution object.
- `template<typename classT, typename T>
void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T class↵
T::*method)`
Binds asynchronously an external action to a class function member.
- `bool is\_running () const`
Checks if this execution has finished.
- `template<typename... Args>
void add_action (actionT action, Args... args)`

6.1.1 Detailed Description

```
template<typename actionT>  
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by using [bind\(\)](#).

Template Parameters

actionT	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
---------	--

6.1.2 Member Function Documentation

6.1.2.1 `bind_action_and_method()`

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method) [inline]
```

Binds asynchronously an external action to a class function member.

The binding is done by using a [bind\(\)](#).

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
obj	- A <code>std::shared_ptr</code> that wraps the bound class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code> .

6.1.2.2 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Checks if this execution has finished.

Returns

true - The execution has not finished.
 false - The execution has finished.

6.1.2.3 `add_action()`

```
template<typename actionT>
template<typename... Args>
void untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```


Template Parameters

Args	
------	--

Parameters

action	
args	

The documentation for this class was generated from the following file:

- [async.hpp](#)

6.2 untangle::async::execution_poll Class Reference

Execution poll class.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
 Adds an [execution](#) object to the poll.
- `auto is_running ()`
 Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
 Gets the single-tone instance.

6.2.1 Detailed Description

Execution poll class.

A single-tone class that may be used to verify if the polled [execution](#) objects have finished their processing. If at least one [execution](#) object is running then this poll has the "running" state.

Remarks

An [execution](#) object runs in a detached thread, hence it is not joinable. The caller can wait for an [execution](#) to be finished only by checking the "running" state([execution::is_running\(\)](#)). This class represents a convenient way to check any number of [execution](#) objects.

Generated by Doxygen

6.2.2 Member Function Documentation

Parameters

<code>async_exec</code>	An execution object.
-------------------------	--------------------------------------

6.2.2.2 `is_running()`

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

6.2.2.3 `get()`

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- [async.hpp](#)

Chapter 7

File Documentation

7.1 `async.hpp` File Reference

Interface to `untangle::async::execution` class.

```
#include <actuator/actuator.hpp>
#include <atomic>
#include <thread>
```

Classes

- class `untangle::async::execution_poll`
Execution poll class.
- class `untangle::async::execution< actionT >`
Asynchronous template execution class.

Functions

- `template<typename classT, typename T, typename actionT = std::function<T>>`
`actionT untangle::async::bind (const std::shared_ptr< classT > &obj, T classT::*method,`
`execution< actionT > &async_exec)`
Asynchronous binding to a class function member.
- `template<typename T, typename actionT = std::function<T>>`
`actionT untangle::async::bind (T &Fn, execution< actionT > &async_exec)`
Asynchronous binding to a plain function.

7.1.1 Detailed Description

Interface to `untangle::async::execution` class.

Author

Nicu Popescu

Date

2018-05-10

7.2 async.hpp

[Go to the documentation of this file.](#)

```

00001
00008 #pragma once
00009
00010 #include <actuator/actuator.hpp>
00011 #include <atomic>
00012 #include <thread>
00013
00014 namespace untangle {
00015     template <typename actionT>
00016     struct actuator;
00017
00018     namespace async {
00019         template <typename actionT>
00020         class execution;
00021     }
00022 } // namespace untangle
00023
00024 namespace untangle {
00025     namespace async {
00026
00029
00046     template <typename classT, typename T, typename actionT = std::function<T>»
00047     actionT bind(const std::shared_ptr<classT>& obj, T classT::* method,
00048                 execution<actionT>& async_exec) {
00049         actionT async_action = untangle::bind(obj, method);
00050
00051         return [&async_exec, async_action](auto... args) -> typename actionT::result_type {
00052             async_exec.add_action(async_action, args...);
00053             return typename actionT::result_type();
00054         };
00055     }
00056
00071     template <typename T, typename actionT = std::function<T>»
00072     actionT bind(T& Fn, execution<actionT>& async_exec) {
00073         actionT async_action = Fn;
00074         return [&async_exec, async_action](auto... args) -> typename actionT::result_type {
00075             async_exec.add_action(async_action, args...);
00076             return typename actionT::result_type();
00077         };
00078     }
00079
00094     class execution_poll {
00095     public:
00101         template <typename asyncexecT>
00102         void add(asyncexecT& async_exec) {
00103             if (!actuator_is_running.is_connected()) {
00104                 actuator_is_running = untangle::connect(async_exec.action_is_running);
00105             } else {
00106                 actuator_is_running.add(&async_exec.action_is_running);
00107             }
00108         }
00109
00115         auto is_running() {
00116             actuator_is_running();
00117
00118             auto result = false;
00119             for (const auto& ret : actuator_is_running.results) {
00120                 result |= ret;
00121             }
00122             return result;
00123         }
00124
00130         static execution_poll& get() {
00131             static execution_poll instance;
00132             return instance;
00133         }
00134
00135     private:
00136         execution_poll() = default;
00137         ~execution_poll() = default;
00138         actuator<std::function<bool(void)>» actuator_is_running;
00139     };
00140
00153     template <typename actionT>
00154     class execution {
00155     public:
00160         execution() {
00161             other_this = this;
00162             action_execute = untangle::bind(other_this, &execution<actionT>::execute_actions);
00163             action_stop = untangle::bind(other_this, &execution<actionT>::stop);
00164             action_is_running = untangle::bind(other_this, &execution<actionT>::is_running);
00165         }
00166         ~execution() = default;

```

```

00167
00178 template <typename classT, typename T>
00179 void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00180                             T classT::* method) {
00181     action = bind(obj, method, *this);
00182 }
00183
00184 template <typename T>
00185 void bind_action_and_function(actionT& action, const T& Fn) {
00186     action = bind(Fn, *this);
00187 }
00188
00195 bool is_running() const { return running.load(); }
00196
00197 void run() {
00198     running = true;
00199     this_thread = std::thread(&execution::execute, this);
00200     this_thread.detach();
00201 }
00202
00203 void start() {
00204     running = true;
00205     this_thread = std::thread(&execution::loop, this);
00206     this_thread.detach();
00207 }
00208
00209 void stop() {
00210     while (!action_list.empty()) {
00211         std::this_thread::sleep_for(std::chrono::milliseconds(10));
00212     }
00213
00214     if (actuator_stop.is_connected()) {
00215         actuator_stop();
00216     }
00217
00218     started = false;
00219 }
00220
00228 template <typename... Args>
00229 void add_action(actionT action, Args... args) {
00230     action_list.push_back(std::bind(action, args...));
00231 }
00232
00233 template <typename otherT>
00234 void attach(otherT& other) {
00235     if (!actuator_execute.is_connected()) {
00236         actuator_execute = untangle::connect(other.action_execute);
00237     } else {
00238         actuator_execute.add(&other.action_execute);
00239     }
00240
00241     if (!actuator_stop.is_connected()) {
00242         actuator_stop = untangle::connect(other.action_stop);
00243     } else {
00244         actuator_stop.add(&other.action_stop);
00245     }
00246 }
00247
00248 auto result() { return _result; }
00249
00250 std::function<void(void)> action_execute;
00251 std::function<void(void)> action_stop;
00252 std::function<bool(void)> action_is_running;
00253 std::function<void(void)> on_finished;
00254 std::string name;
00255
00256 private:
00257     // SFINAE by return (void)
00258     template <typename T = actionT>
00259     typename std::enable_if_t<std::is_void<typename T::result_type>::value, typename T::result_type>
00260     select_execute_actions() {
00261         action_list.front();
00262     }
00263
00264     // SFINAE by return (non-void)
00265     template <typename T = actionT>
00266     typename std::enable_if_t<!std::is_void<typename T::result_type>::value, typename T::result_type>
00267     select_execute_actions() {
00268         _result = action_list.front();
00269     }
00270
00271     return typename T::result_type();
00272 }
00273
00274 void execute_actions() {
00275     while (!action_list.empty()) {
00276         select_execute_actions();
00277     }

```

```

00277     action_list.pop_front();
00278 }
00279
00280 if (actuator_execute.is_connected()) {
00281     actuator_execute();
00282 }
00283 }
00284
00285 void execute() {
00286     execute_actions();
00287
00288     if (on_finished) {
00289         on_finished();
00290     }
00291
00292     std::cout << "finishing thread" << std::endl;
00293
00294     running = false;
00295 }
00296
00297 void loop() {
00298     started = true;
00299
00300     while (started.load()) {
00301         execute_actions();
00302     }
00303
00304     running = false;
00305
00306     std::cout << "thread finished" << std::endl;
00307 }
00308
00309 std::thread this_thread;
00310
00311 std::atomic_bool started = {false};
00312 std::atomic_bool running = {false};
00313
00314 std::list<std::function<typename actionT::result_type(void)>> action_list;
00315
00316 actuator<std::function<void(void)>> actuator_execute;
00317 actuator<std::function<void(void)>> actuator_stop;
00318
00319 // std::vector cannot hold void type; use an arbitrary type e.g. int
00320 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00321                                 typename actionT::result_type>;
00322 typename resultT::type __result;
00323
00324 execution<actionT>* other_this;
00325 };
00326 } // namespace async
00327 } // namespace untangle

```

Index

- add
 - untangle::async::execution_poll, [13](#)
- add_action
 - untangle::async::execution< actionT >, [12](#)
- async, [1](#)
- async.hpp, [15](#)
- bind
 - namespace untangle: functions, [9](#), [10](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [12](#)
- get
 - untangle::async::execution_poll, [14](#)
- is_running
 - untangle::async::execution< actionT >, [12](#)
 - untangle::async::execution_poll, [14](#)
- namespace untangle: functions, [9](#)
 - bind, [9](#), [10](#)
- untangle::async::execution< actionT >, [11](#)
 - add_action, [12](#)
 - bind_action_and_method, [12](#)
 - is_running, [12](#)
- untangle::async::execution_poll, [13](#)
 - add, [13](#)
 - get, [14](#)
 - is_running, [14](#)